

week02-d...Steps to cr...UserStorie...Kaggadas...amazon -...Online Sh...Extensions...Flipkart P...Online Sh...

https://www.amazon.in/?tag=msndeskstdin-21&ref=pd_sl_5gd5fvp5cw_b&adgrpid=1320515864101687&hvadid=82532470830494&hvnetw...

Microwaves

Washing machines

Home tools

Wallpapers

Min. 50% off | Office furniture

Registe Udyam

See more

Explore all

Create a free account

Up to 40% off | Headphones and earbuds

See all offers



Up to 40% Off | Deals on trendy furniture

See all offers



https://www.amazon.in/Sony-HT-S20R-Soundbar-Bluetooth-Connectivity/dp/B084685MT1/?_encoding=UTF8&pd_rd_w=PTICf&conten...

AI Code Generator

Code Generator

Settings

☒ Feature File

☒ Page Class

Reset Chat

Provide additional information

Stop

Generate

Type here to search



ENG

16:47

17-04-2026

AI Code Generator

Code Generator

Settings

Language Binding

Java

Browser Engine

Selenium

Select LLM Provider

Groq

Select Model

openai/gpt-oss-120b

Groq API Key

.....

Feature: Verify product image on Amazon India

As a QA engineer,

I want to search for a product on Amazon.in and verify that the displayed product image contains the correct alt-text, so that the correct item is shown to the user.

Scenario Outline: Search for a product and verify its image alt attribute

Given I open the Amazon India homepage

When I search for "<product>"

And I click on the first product result that contains "<product>"

Then the product image alt attribute should contain "<expectedAlt>"

Examples:

product	expectedAlt	
Sony HT-S20R	Sony HT-S20R Real 5.1ch Dolby Digital Soundbar for TV with subwoofer and Compact Rear Speakers	
OnePlus 11	OnePlus 11 (India) 5G Smartphone with Snapdragon 8 Gen 2, 16GB RAM, 256GB Storage	
Apple iPhone 14	Apple iPhone 14 (128GB) - Midnight	

```
package com.amazon.stepdefs;
```

```
import io.cucumber.java.After;
```

```
import io.cucumber.java.Before;
```

```
import io.cucumber.java.en.*;
```

```
import org.junit.Assert;
```

```
import org.openqa.selenium.*;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.openqa.selenium.support.ui.*;
```

```
import java.time.Duration;
```

```
import java.util.List;
```

```
public class ProductImageStepDefs {
```

```
private WebDriver driver;  
private WebDriverWait wait;  
private static final String AMAZON_URL = "https://www.amazon.in/";
```

@Before

```
public void setUp() {  
    // Assuming chromedriver is on the system PATH.  
    driver = new ChromeDriver();  
    wait = new WebDriverWait(driver, Duration.ofSeconds(15));  
    driver.manage().window().maximize();  
}
```

@After

```
public void tearDown() {  
    if (driver != null) {  
        driver.quit();  
    }  
}
```

@Given("I open the Amazon India homepage")

```
public void openAmazonHomePage() {  
    driver.get(AMAZON_URL);  
    // Wait for the search box to be present to ensure page load.
```

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("twotabsearchtextbox")));  
}
```

```

@When("I search for {string}")
public void searchForProduct(String product) {
    WebElement searchBox = wait.until(
        ExpectedConditions.elementToBeClickable(By.id("twotabsearchtextbox")));
    searchBox.clear();
    searchBox.sendKeys(product);

    WebElement searchBtn = driver.findElement(By.id("nav-search-submit-button"));
    searchBtn.click();

    // Wait for the results page to load.
    wait.until(ExpectedConditions.visibilityOfElementLocated(
        By.cssSelector("div.s-main-slot")));
}

```

```

@When("I click on the first product result that contains {string}")
public void clickFirstResult(String product) {
    // Locate all product title elements that contain the search term.
    List<WebElement> titles = wait.until(
        ExpectedConditions.presenceOfAllElementsLocatedBy(
            By.xpath("//span[contains(@class,'a-size-medium') and contains(text(),'"+
product + "')]")));

    if (titles.isEmpty()) {
        Assert.fail("No product titles containing '" + product + "' were found.");
    }

    // Click the first matching title.

```

```

WebElement firstTitle = titles.get(0);

// The clickable element is the ancestor <a> tag.
WebElement productLink = firstTitle.findElement(By.xpath("./ancestor::a"));
wait.until(ExpectedConditions.elementToBeClickable(productLink)).click();


// Switch to the new tab if Amazon opened the product in a new window.
for (String handle : driver.getWindowHandles()) {
    driver.switchTo().window(handle);
}


// Wait for the product image to be visible.
wait.until(ExpectedConditions.visibilityOfElementLocated(
    By.cssSelector("img#landingImage, img[data-a-dynamic-image]")));
}


@Then("the product image alt attribute should contain {string}")
public void verifyImageAlt(String expectedAlt) {
    // Amazon may use different selectors for the main image; try a few.
    By[] imageLocators = new By[]{
        By.id("landingImage"),
        By.cssSelector("img[data-a-dynamic-image]"),
        By.cssSelector("img.s-image")
    };

    WebElement img = null;
    for (By locator : imageLocators) {
        try {
            img = wait.until(ExpectedConditions.visibilityOfElementLocated(locator));

```

```
        if (img != null && img.isDisplayed()) {  
            break;  
        }  
    } catch (TimeoutException ignored) {  
        // Try next locator  
    }  
}
```

```
Assert.assertNotNull("Product image was not found on the page.", img);
```

```
String altText = img.getAttribute("alt");
```

```
Assert.assertNotNull("Alt attribute is missing on the product image.", altText);
```

```
Assert.assertTrue(  
    "Alt text does not contain expected value.
```

```
Expected to contain: "" + expectedAlt + ""
```

```
Actual: "" + altText + """,
```

```
    altText.contains(expectedAlt));
```

```
    }
```

```
}
```